GitHub Copilot 워크플로 과정

GitHub Copilot 실무 활용 과정

강사 소개 · 강의 개요 · 실습 환경 설정

1 강사 소개

2 강의 개요

3 실습 환경 설정

강사 소개

이동준 — 10년차 개발자 · (현) 카카오 AI Platform Backend Engineer



1 **Conference**

ifkakao, 우아한 테크세미나

2 **Lecture**

Fastcampus, 숙명여대 특강, 광주영상복합문화관 특강

3 **Mentoring**

SWMaestro, F-Lab, Codeit,
SK Networks, Kakao tech,
Grepp, Sessac

01

강의 개요

프로젝트 하나를 이슈 단위로 개발하며 Copilot 기능을 순서대로 써 봅니다.

모듈 구성

모듈	주제	해 보는 것
M0 시작하기	과정 소개, 실습 환경	내 repo 만들고 실행
M1 공통 정리	실행 모드 · 환경 · 토큰 · 모델	Skill 로 흐름 파악, 비용 비교
M2 Instructions	instructions 세 종류 · 적용 범위	초안 → 다이어트 · 경로별 2개
M3 Skills · Agents	Skill · 커스텀 에이전트	alarm-rule Skill 로 이슈 #1, 리뷰어
M4 사용량 · Harness	사용량 · 낭비 패턴 · Hooks	처방 비교, Hook, 대화 리셋
M5 MCP & 보안	MCP 연결 · 보안	이슈 등록 · #2 구현, 인젝션 체험
M6 PR · Review · CI	요구사항 → CI/CD → PR → 리뷰	CI 수정, 리뷰 반영, 자동 배포 · Done
M7 만들고 공유하기	배운 것으로 직접 만들기	주제 하나 완성 → 3분 공유

진행 방식

1. **개념 → 바로 실습**
   개념을 본 뒤 그 자리에서 직접 해 보기
2. **프로젝트 하나로 끝까지**
   `equip-monitor` 를 이슈 단위로 개발
3. **체크포인트 브랜치**
   뒤처지면 `m3-start ~ m6-start` 로 합류
4. **Python · C#**
   내용 동일, 강사 데모는 Python
5. **마지막은 직접**
   M7 — 배운 것으로 내 주제 하나 만들고 공유
6. **사용량**
   1인 요청 60건 안팎

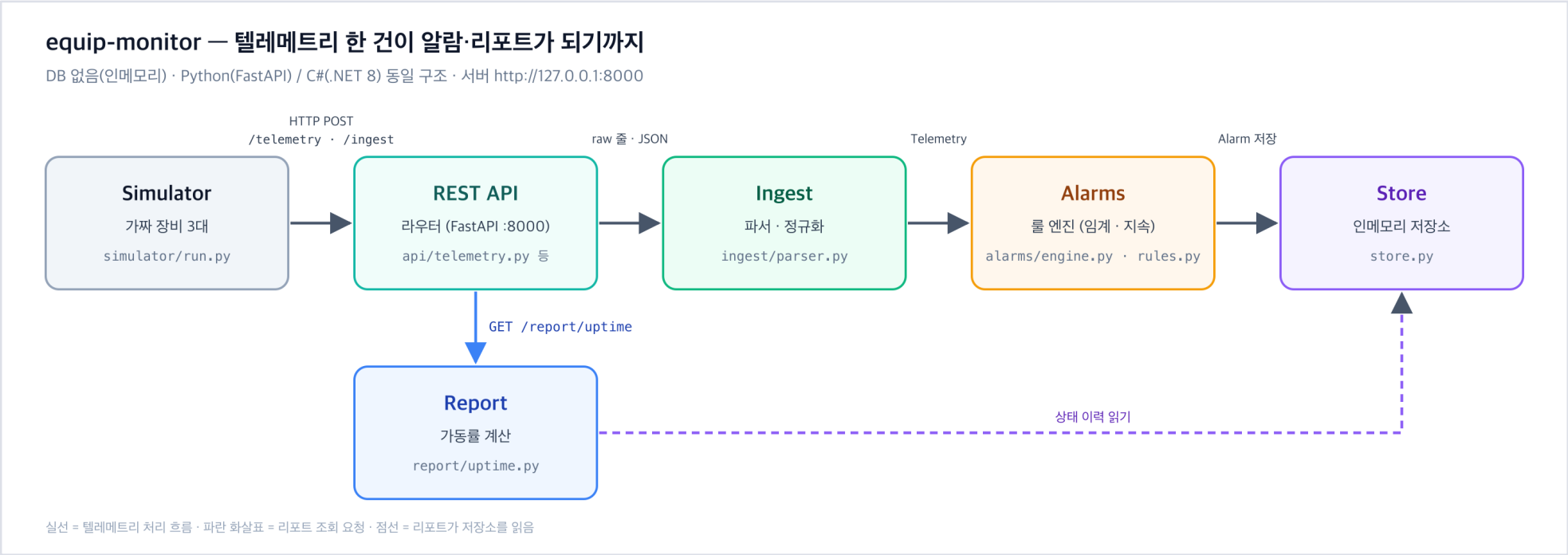
02

실습 환경 설정

템플릿으로 내 repo를 만들고 앱을 실행합니다. 이후 모든 모듈이 이 repo에서 진행됩니다.

실습 프로젝트 — equip-monitor

장비 텔레메트리 → 알람 → 가동률. DB 없이 10분이면 전체를 읽는 크기입니다.



1 Simulator → API

가짜 장비 3대 → `/telemetry`, `/ingest` 로 POST

2 Ingest → Alarms

`parser.py` 정규화, `engine.py`·`rules.py` 룰 평가

3 Store · Report

인메모리 저장, 가동률 계산 (API는 이슈 #2)

docs · scripts 폴더

코드 밖에서 실습에 쓰는 두 폴더입니다.

폴더	파일	하는 일
docs/issues/	01-sustained-alarm.md	85℃ 5분 지속 경고 — 요구사항
docs/issues/	02-uptime-report.md	장비별 가동률 리포트 API — 요구사항
docs/issues/	03-log-format-question.md	프롬프트 인젝션 실습용 — 숨은 지시 포함, 따르지 않기
scripts/	check_env.py	실습 환경 점검 (곧 실행)
scripts/	check_rules.py	로그 파일을 룰에 흘려 발생 알람을 표로 출력

C#: `scripts\check_env.ps1`, 알람 표는 `src/EquipMonitor.CheckRules`

docs/issues/ — GitHub 이슈로 구현할 요구사항 2개

문서마다 배경 + 인수조건(체크리스트). Copilot에게 구현을 맡길 때 이 문서를 그대로 줍니다.

01-sustained-alarm.md

85°C 5분 지속 경고

새 룰 `temp-sustained` — 85°C 이상 300초 지속 시 WARNING 1건, 회복 후 다시 지속되면 또 1건

02-uptime-report.md

가동률 리포트 API

`GET /report/uptime` — 기간 · 장비별 상태 시간과 가동률, 없는 장비는 404

실습

내 repo 만들고 실행하기

템플릿으로 내 repo를 만들고, 환경 점검 → 테스트 → 앱 실행까지 확인합니다.
이후 모든 모듈이 이 repo에서 진행됩니다.

준비

- GitHub 로그인된 브라우저
- VS Code · Python 3.11+ (또는 .NET 10)
- 터미널 2개

실습 1

템플릿으로 내 repo 만들기

1. 템플릿 → **Use this template** ▾ → **Create a new repository**
2. **Include all branches** 를 **On** (기본 Off)
3. **Owner** → **HWM-RND** 선택
4. **Repository name** 끝에 `-<본인 ID>` → `… is available` · **Private**
5. Jumpstart 칸은 비우고 **Create repository** (멈추면 **Refresh**)

템플릿 · Owner · 이름

```
# 템플릿 repo — Python
github.com/dongjoon1251/equip-monitor-py

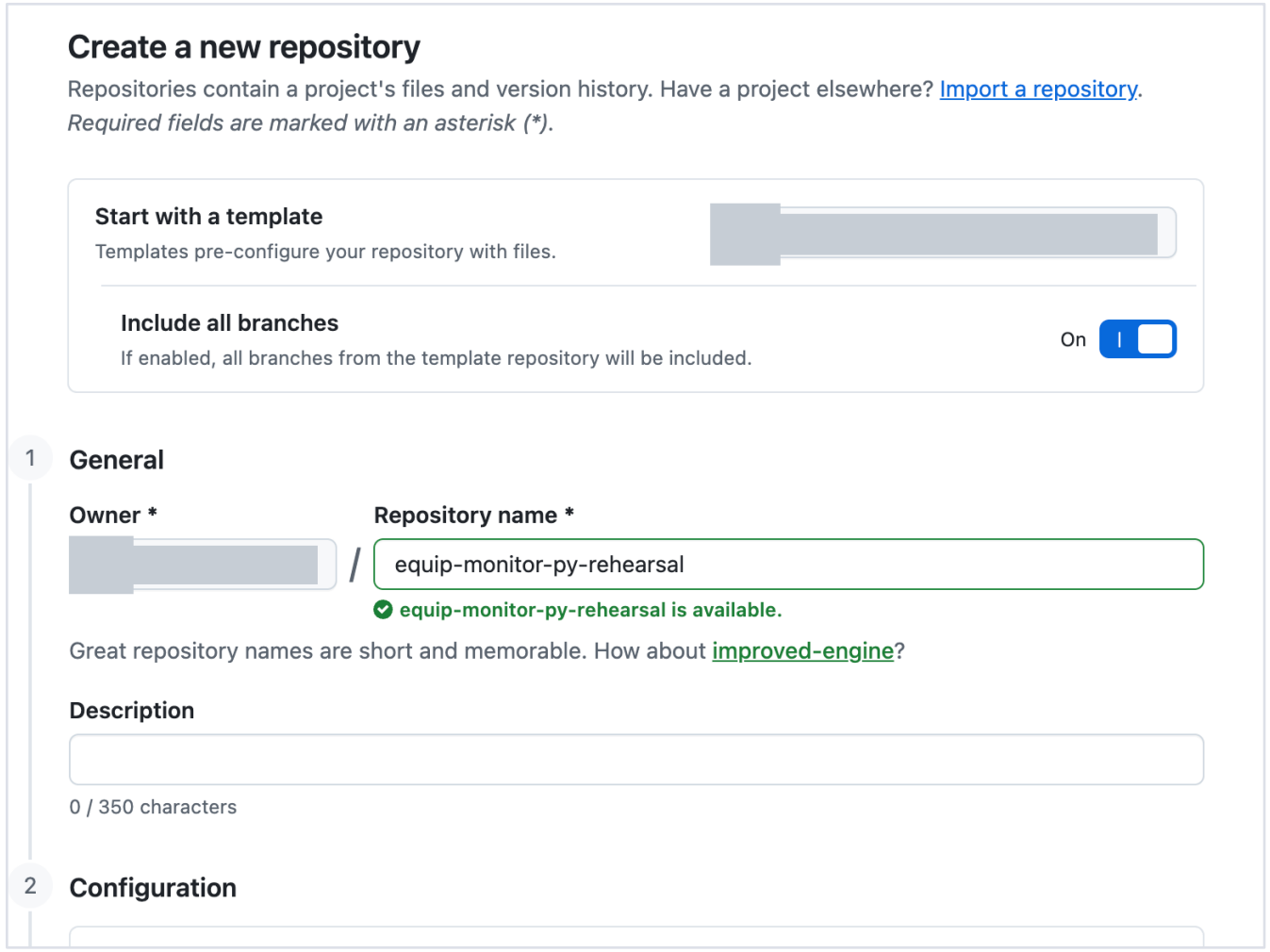
# 템플릿 repo — C#
github.com/dongjoon1251/equip-monitor-cs

# Owner
HWM-RND

# Repository name (예: 본인 ID hong01)
equip-monitor-py-hong01
equip-monitor-cs-hong01
```

✅ **5 Branches** ✅ **`main · m3-start ~ m6-start`**

생성 화면 — Include all branches



1 **Include all branches → On**

체크포인트 브랜치 4개가 함께 복사

2 **Owner · Repository name**

HWM-RND · 이름 끝에 본인 ID

3 **Choose visibility**

Private 선택

실습 2

clone + 환경 점검

1. 내 repo를 clone 하고 venv 생성
2. venv 활성화 (오른쪽 `# venv 활성화` 줄)
3. 의존성 설치 후 `check_env` 실행
4. `python -m pytest -q` 로 테스트 확인
5. C#: `scripts\check_env.ps1` 점검 → `dotnet test` (명령은 가이드)

명령 (Python)

```
git clone https://github.com/HWM-RND/<repo>.git
cd <repo>
python -m venv .venv

# venv 활성화 — Windows / mac·Linux
.venv\Scripts\Activate.ps1
source .venv/bin/activate

python -m pip install -r requirements.txt
python scripts/check_env.py
python -m pytest -q
```

✅ 마지막 줄 모두 OK ✅ 26 passed (C# 27 passed)

check_env 결과

```
$ python scripts/check_env.py
[OK ] Python 3.12.11 >= 3.11
[OK ] import fastapi
[OK ] import uvicorn
[OK ] import httpx
[OK ] import pytest
[OK ] import mcp
[OK ] import equip_monitor
[OK ] git on PATH
[OK ] gh on PATH
[OK ] code on PATH
[WARN] gh auth status   → (선택) gh auth login — 이슈 생성(seed)·PR 생성에 사용, 안 되면 GitHub 웹에서 직접 해도 됩니다
[OK ] GitHub Copilot Chat (확장 또는 VS Code 내장)

모두 OK — 이 화면을 캡처해 제출하세요.
((.venv) ) ~/lectures/copilot/equip-monitor-py $
```

1 **[OK] 줄**

Python 버전 · 패키지 · git · code · Copilot Chat

2 **[WARN] gh auth status**

선택 항목 — 통과에 영향 없음

3 **마지막 줄**

모두 OK — 이 화면을 캡처해 제출하세요.

샘플 데이터 — data/samples/

한 줄이 텔레메트리 1건: 시각 장비 `state=…` `temp=…` `pressure=…` (# 줄은 설명)

파일	내용	지금 코드로 돌리면
`normal.log`	정상 가동 — 3대, 1분 간격 15건	알람 0건
`overheat.log`	DEV-02 과열 — 85°C 이상이 두 번 이어지다 09:16 에 96.5°C	`temp-critical` 1건 (09:16)

overheat.log — 과열 시나리오

data/samples/overheat.log

```
# DEV-02 과열 시나리오. 이슈 #1 (85°C 5분 지속) 기대 알람: 09:07, 09:15
2026-09-21T09:00:00Z DEV-01 state=RUN temp=61.0 pressure=1.01
2026-09-21T09:00:00Z DEV-02 state=RUN temp=70.0 pressure=1.10
2026-09-21T09:01:00Z DEV-01 state=RUN temp=61.4 pressure=1.01
2026-09-21T09:01:00Z DEV-02 state=RUN temp=84.9 pressure=1.20
2026-09-21T09:02:00Z DEV-01 state=RUN temp=61.2 pressure=1.02
2026-09-21T09:02:00Z DEV-02 state=RUN temp=86.0 pressure=1.30
2026-09-21T09:03:00Z DEV-02 state=RUN temp=86.5 pressure=1.35
2026-09-21T09:04:00Z DEV-02 state=RUN temp=87.0 pressure=1.40
2026-09-21T09:05:00Z DEV-01 state=RUN temp=61.9 pressure=1.02
2026-09-21T09:05:00Z DEV-02 state=RUN temp=87.2 pressure=1.42
2026-09-21T09:06:00Z DEV-02 state=RUN temp=87.8 pressure=1.45
2026-09-21T09:07:00Z DEV-02 state=RUN temp=88.1 pressure=1.50
2026-09-21T09:08:00Z DEV-02 state=RUN temp=88.0 pressure=1.50
2026-09-21T09:09:00Z DEV-02 state=IDLE temp=70.0 pressure=1.00
2026-09-21T09:10:00Z DEV-02 state=RUN temp=86.2 pressure=1.30
2026-09-21T09:11:00Z DEV-02 state=RUN temp=86.9 pressure=1.33
2026-09-21T09:12:00Z DEV-02 state=RUN temp=87.4 pressure=1.36
2026-09-21T09:13:00Z DEV-02 state=RUN temp=88.0 pressure=1.40
2026-09-21T09:14:00Z DEV-02 state=RUN temp=88.6 pressure=1.44
2026-09-21T09:15:00Z DEV-02 state=RUN temp=89.0 pressure=1.48
2026-09-21T09:16:00Z DEV-02 state=RUN temp=96.5 pressure=1.60
```

1. **줄 형식**

   시각 · 장비 · `state` · `temp` · `pressure`

2. **DEV-02 가 85°C 이상**

   09:02~09:08, 09:10~09:15 두 구간

3. **09:16 `temp=96.5`**

   95°C 이상 → `temp-critical`

normal.log — 정상 가동

data/samples/normal.log

```
# 정상 가동. 3대, 1분 간격
2026-09-21T09:00:00Z DEV-01 state=RUN temp=61.2 pressure=1.01 vibration=0.30
2026-09-21T09:00:00Z DEV-02 state=RUN temp=58.9 pressure=0.98 vibration=0.22
2026-09-21T09:00:00Z DEV-03 state=IDLE temp=25.0 pressure=1.00 vibration=0.01
2026-09-21T09:01:00Z DEV-01 state=RUN temp=61.8 pressure=1.02 vibration=0.31
2026-09-21T09:01:00Z DEV-02 state=RUN temp=59.4 pressure=0.99 vibration=0.25
2026-09-21T09:01:00Z DEV-03 state=IDLE temp=25.1 pressure=1.00 vibration=0.01
2026-09-21T09:02:00Z DEV-01 state=RUN temp=62.0 pressure=1.02 vibration=0.29
2026-09-21T09:02:00Z DEV-02 state=RUN temp=60.1 pressure=1.00 vibration=0.27
2026-09-21T09:02:00Z DEV-03 state=RUN temp=40.3 pressure=1.05 vibration=0.20
2026-09-21T09:03:00Z DEV-01 state=RUN temp=61.9 pressure=1.01 vibration=0.30
2026-09-21T09:03:00Z DEV-02 state=DOWN temp=55.0 pressure=0.90 vibration=0.00
2026-09-21T09:03:00Z DEV-03 state=RUN temp=52.7 pressure=1.08 vibration=0.24
2026-09-21T09:04:00Z DEV-01 state=RUN temp=62.3 pressure=1.02 vibration=0.32
2026-09-21T09:04:00Z DEV-02 state=DOWN temp=50.2 pressure=0.90 vibration=0.00
2026-09-21T09:04:00Z DEV-03 state=RUN temp=58.8 pressure=1.10 vibration=0.26
```

장비 3대 · 1분 간격 · `state` 가 RUN · IDLE · DOWN 으로 바뀜 — 알람 0건

실습 3

앱 실행 + 시뮬레이터 리플레이

1. 터미널 ①: 서버 실행
2. 터미널 ②: 과열 로그 재생
3. `/docs` 열기 — devices · telemetry · alarms
4. **GET /alarms → Try it out → Execute**

명령 (Python)

```
# 터미널 ① 서버
python -m uvicorn equip_monitor.app:app --
reload

# 터미널 ② 과열 로그 재생
python simulator/run.py --replay
data/samples/overheat.log

# 브라우저
http://127.0.0.1:8000/docs
```

✅ `Uvicorn running on …:8000` ✅ `202 … 'accepted': 21` ✅ `temp-critical 1건`

GET /alarms 결과

200

Response body


```
[
  {
    "id": 1,
    "device_id": "DEV-02",
    "rule": "temp-critical",
    "severity": "CRITICAL",
    "message": "temp=96.5 >= 95",
    "ts": "2026-09-21T09:16:00Z",
    "acked": false
  }
]
```


1. **`rule: temp-critical`**
   과열 로그가 만든 알람 1건

2. **`severity: CRITICAL`**
   `temp=96.5 >= 95` 임계값 초과

3. **`ts` 가 `09:16:00Z`**
   리플레이 로그 시각 그대로

핵심 요약

1. **HWM-RND** 에 …-<본인 ID> 이름으로, **Include all branches** 켜고 생성
2. `check_env` 마지막 줄 모두 `OK`, 테스트 `26 passed` (C# 27)
3. 서버 실행 → 과열 로그 리플레이 → `temp-critical` 알람 1건
4. 뒤처지면 `m3-start ~ m6-start` 브랜치로 합류